

Introduction to Software Architecture

Monolithic Architecture

Name	Kishore G
Roll No.	26MCT010
Programme	M.Sc Computer Technology
Department	Information Technology and Cognitive Systems
College	Sri Krishna College of Arts and Science

1. Introduction to Software Architecture

Software architecture is the high-level structure of a software system. It defines how a system is broken into parts, how those parts are expected to communicate with each other, and the rules that govern how the system as a whole is built, deployed and maintained. It is easiest to think of it as the equivalent of a blueprint for a building. A blueprint does not describe the colour of the paint or the exact position of every piece of furniture, but it does decide where the load-bearing walls go, how many floors the building has, and how people move between rooms. In the same way, software architecture does not describe every line of code, but it does decide how the application is organised and how its major parts fit together.

Choosing an architecture matters because it affects almost everything that happens after the first version of a system is built: how easy it is to add a new feature, how easy it is to find and fix a bug, how the system behaves under heavy load, and how many different teams can work on it at the same time without stepping on each other. Over the years, several architectural styles have become common, including layered architecture, monolithic architecture, microservices architecture, and event-driven architecture. This document focuses on one of the oldest and still most widely used styles: monolithic architecture.

2. What Is Monolithic Architecture

A monolithic architecture is one where an entire application is built and shipped as a single, unified unit. Every part of the system, including the user interface, the business logic that makes decisions, and the code that talks to the database, is compiled together and deployed together. There is only one codebase and, in most cases, only one running process at a time. This does not mean the code inside is disorganised. A well-written monolith is usually still split internally into layers or modules, such as a presentation layer, a service layer and a data access layer. The key difference from other styles is that these layers all live inside the same application and are deployed as one piece, rather than being separate, independently deployable services.

The word "monolithic" comes from the idea of a single large block of stone. It is a fitting description, since a monolithic application behaves like one solid block: to change even a small part of it, you typically rebuild and redeploy the entire application, not just the part that changed.

3. Characteristics of Monolithic Architecture

Characteristic	What it means in practice
Single codebase	All source code lives in one project and one repository
Single deployment unit	The whole application is packaged as one .jar, .war or executable
Shared database	All modules typically read and write to the same database
In-process communication	Modules call each other directly through method calls, not over a network
Shared runtime	Every module runs inside the same process and memory space

These characteristics together explain why monolithic applications are often described as simple to reason about. Since everything lives in one place, a developer can usually trace a request from the moment it enters the system to the moment a response is sent back, all within the same codebase, without needing to jump across separate services or worry about network calls failing along the way.

4. Typical Internal Structure

Even though a monolith is deployed as one unit, good practice still calls for internal separation of concerns. A common approach, and the one used in the example later in this document, splits the application into three layers.

4.1 Presentation Layer

This is the part the user directly interacts with, such as a Swing window, a web page, or a REST controller. Its job is limited to collecting input and displaying output; it should not contain business rules.

4.2 Business Logic Layer

Often called the service layer, this is where decisions are made: validating data, applying rules, and coordinating what should happen for a given action. This layer does not know or care whether the request came from a desktop window or a web page.

4.3 Data Access Layer

This layer is responsible for reading and writing data, whether that data lives in a database, a file, or, as in the simplified example below, an in-memory list standing in for a database table. Keeping this layer separate means the storage mechanism can be changed later without having to rewrite the business logic.

The important thing to notice is that all three layers above still end up inside the same project, are compiled together, and are shipped as a single deployable artifact. That single deployment step is what makes the architecture monolithic, not the absence of internal structure.

5. Advantages and Disadvantages

5.1 Advantages

Monolithic applications are simple to develop in the early stages of a project, since there is only one codebase to set up and one application to run locally. They are also simple to deploy, because a single build produces a single artifact that can be handed over and run. Testing tends to be more straightforward too, since everything happens inside one process without having to simulate calls between separate services. Performance can also be an advantage, since calls between modules are ordinary in-process method calls rather than network requests, which avoids the delay that comes with network communication.

5.2 Disadvantages

As an application grows, a monolith can become harder to work with. The codebase tends to grow large, which makes it harder for new developers to understand it quickly. Because every module is

bundled together, even a small change to one part of the system usually requires rebuilding and redeploying the entire application. Scaling is also less flexible: if only one part of the application, such as report generation, needs more computing power, the entire application still has to be scaled as one unit, since there is no way to scale a single module on its own. Finally, because all modules share the same codebase and often the same database, changes made carelessly in one part of the system can unexpectedly affect another part.

6. A Monolithic Application in Java

The example below is a small Library Management System. It is intentionally kept simple, but it is structured the way a real monolith would be: a data access layer that stands in for a database, a service layer that contains the actual business rules, and a presentation layer that a user interacts with. Every one of these classes belongs to the same project and is compiled into the same program, which is what makes this a monolithic design rather than a distributed one.

6.1 Model

```
public class Book {
    private String isbn;
    private String title;
    private boolean issued;

    public Book(String isbn, String title) {
        this.isbn = isbn;
        this.title = title;
        this.issued = false;
    }

    public String getIsbn() { return isbn; }
    public String getTitle() { return title; }
    public boolean isIssued() { return issued; }
    public void setIssued(boolean issued) { this.issued = issued; }
}
```

The Book class represents a single record used across every layer.

6.2 Data Access Layer

In a full application, this class would talk to a real database using JDBC or a framework such as Hibernate. Here, an in-memory list plays that role, but the important part is that the rest of the application only talks to this class, and never touches the underlying storage directly.

```
import java.util.*;

public class BookDAO {
    private final Map<String, Book> books = new HashMap<>();

    public void save(Book book) {
        books.put(book.getIsbn(), book);
    }

    public Book findByIsbn(String isbn) {
        return books.get(isbn);
    }

    public List<Book> findAll() {
        return new ArrayList<>(books.values());
    }
}
```

BookDAO handles storage and retrieval; it contains no business rules.

6.3 Business Logic Layer

The service class is where the actual rules of the library live, such as refusing to issue a book that is already issued. This is the layer the presentation code should call into; it should never bypass this layer and go straight to the DAO.

```
public class LibraryService {
    private final BookDAO bookDAO;

    public LibraryService(BookDAO bookDAO) {
        this.bookDAO = bookDAO;
    }

    public void addBook(String isbn, String title) {
        bookDAO.save(new Book(isbn, title));
    }

    public String issueBook(String isbn) {
        Book book = bookDAO.findByIsbn(isbn);
        if (book == null) {
            return "No book found with that ISBN.";
        }
        if (book.isIssued()) {
            return "This book is already issued.";
        }
        book.setIssued(true);
        return "Book issued successfully: " + book.getTitle();
    }

    public String returnBook(String isbn) {
        Book book = bookDAO.findByIsbn(isbn);
        if (book == null) {
            return "No book found with that ISBN.";
        }
        book.setIssued(false);
        return "Book returned successfully: " + book.getTitle();
    }
}
```

LibraryService contains the rules that decide what is allowed to happen.

6.4 Presentation Layer

The presentation layer is a small console menu here, kept deliberately simple so that the focus stays on the architecture rather than on interface design. In a real project this could just as easily be the Swing interface covered in the earlier assignment, since the same service and DAO classes underneath would not need to change.

```
import java.util.Scanner;

public class LibraryApp {
    public static void main(String[] args) {
        BookDAO bookDAO = new BookDAO();
        LibraryService service = new LibraryService(bookDAO);
        Scanner scanner = new Scanner(System.in);

        service.addBook("101", "Introduction to Algorithms");
        service.addBook("102", "Effective Java");

        while (true) {
            System.out.println("\n1. Issue Book 2. Return Book 3. Exit");
            System.out.print("Choose an option: ");
            String choice = scanner.nextLine();
        }
    }
}
```

```

        if (choice.equals("1")) {
            System.out.print("Enter ISBN to issue: ");
            System.out.println(service.issueBook(scanner.nextLine()));
        } else if (choice.equals("2")) {
            System.out.print("Enter ISBN to return: ");
            System.out.println(service.returnBook(scanner.nextLine()));
        } else if (choice.equals("3")) {
            break;
        }
    }
    scanner.close();
}
}

```

LibraryApp is the entry point; it is also where all three layers meet.

Everything above, the model, the data access layer, the service layer and the console interface, gets compiled into a single program and run as one process. There is no separate deployment for the data access code and no separate server for the business logic. That single, combined deployment is the entire point of a monolithic architecture, and it is what distinguishes this design from a microservices version of the same idea, where the book catalogue and the issuing logic might instead run as two separate services communicating over a network.

7. When Monolithic Architecture Makes Sense

Monolithic architecture is not an outdated choice; it is simply better suited to some situations than others. It tends to work well for small to medium applications, for teams that are just starting out and want to move quickly, and for systems where the extra complexity of running multiple independent services is not yet justified. Many successful applications are launched as a monolith and only split into smaller services later, once a specific part of the system genuinely needs to scale or be worked on independently. Starting with a monolith and evolving it later is often a more practical path than starting with a complex distributed system before there is a real need for one.

8. Monolithic Architecture Compared with Microservices

It is easier to appreciate what monolithic architecture is once it is placed next to its most common alternative: microservices architecture. In a microservices design, the same application would be broken into several small, independently deployable services, for example a separate service for the book catalogue and another for issuing and returns, each with its own codebase and often its own database, communicating with each other over the network. The table below summarises the main differences.

Aspect	Monolithic	Microservices
Deployment	One unit, deployed together	Many units, deployed independently
Codebase	Single, shared codebase	Separate codebase per service
Communication	In-process method calls	Network calls between services
Scaling	Whole application scales as one	Each service scales on its own
Initial complexity	Low; easy to start with	Higher; more moving parts to set up

Best suited for	Small to medium applications, early-stage projects	Large systems with independent teams and workloads
-----------------	--	--

Neither style is universally better than the other; they solve different problems. Microservices exist mainly to address the scaling and organisational difficulties that show up once a monolith has grown very large and is worked on by many teams at once. For a project of the size covered in this document, a monolith is the more sensible starting point, since it avoids the extra setup and network overhead that microservices bring without giving any real benefit at this scale.

9. Conclusion

Monolithic architecture keeps an entire application, from the user interface down to data storage, inside a single deployable unit. This makes it straightforward to build, test and deploy, especially early in a project's life, though it can become harder to manage as the application and the team working on it both grow. The Library Management System built in this document shows that a monolith does not mean messy code; internal layering into presentation, business logic and data access still applies, and is still good practice. What defines the architecture as monolithic is simply that all of these layers are packaged and deployed together as one piece.